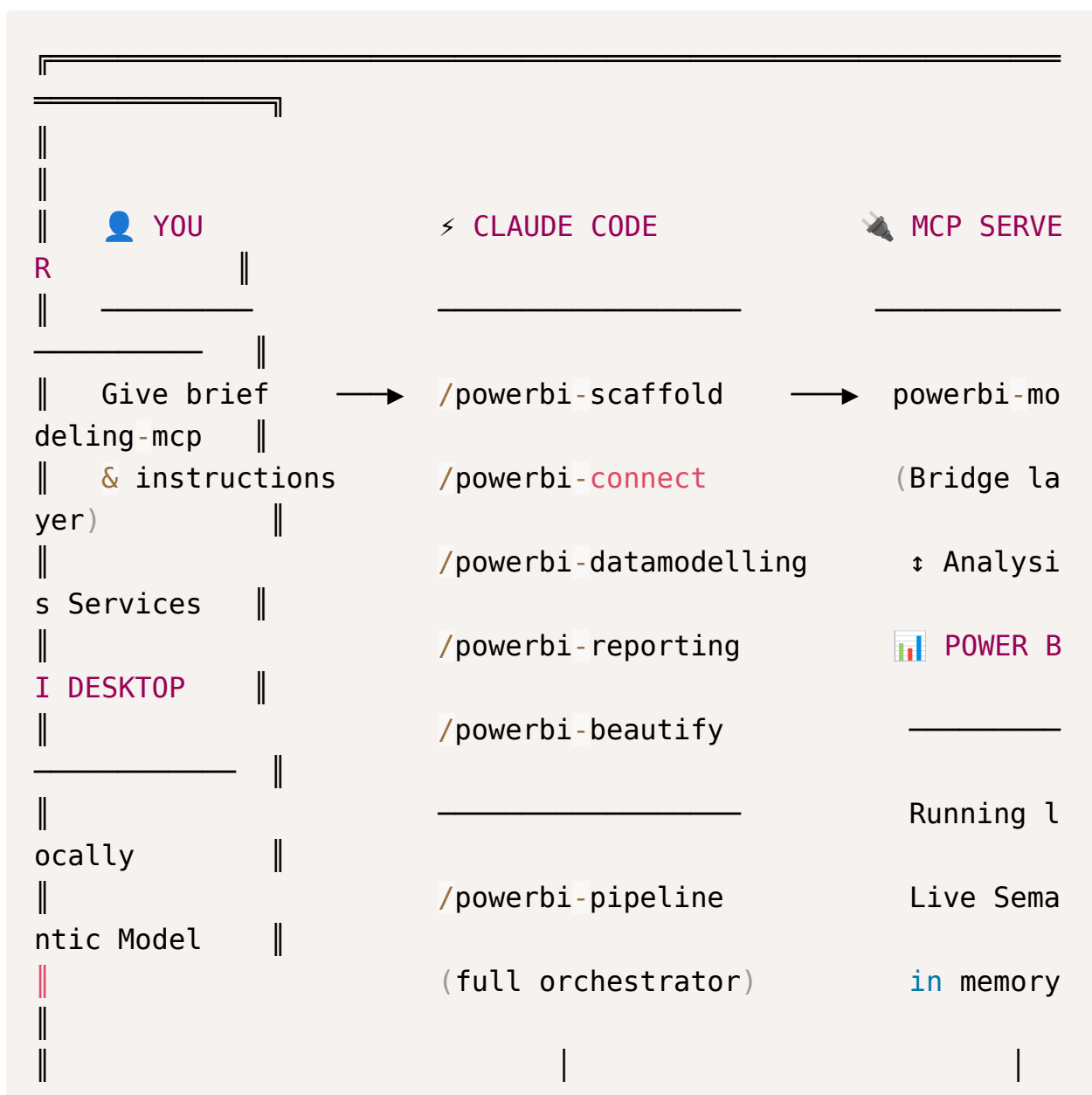
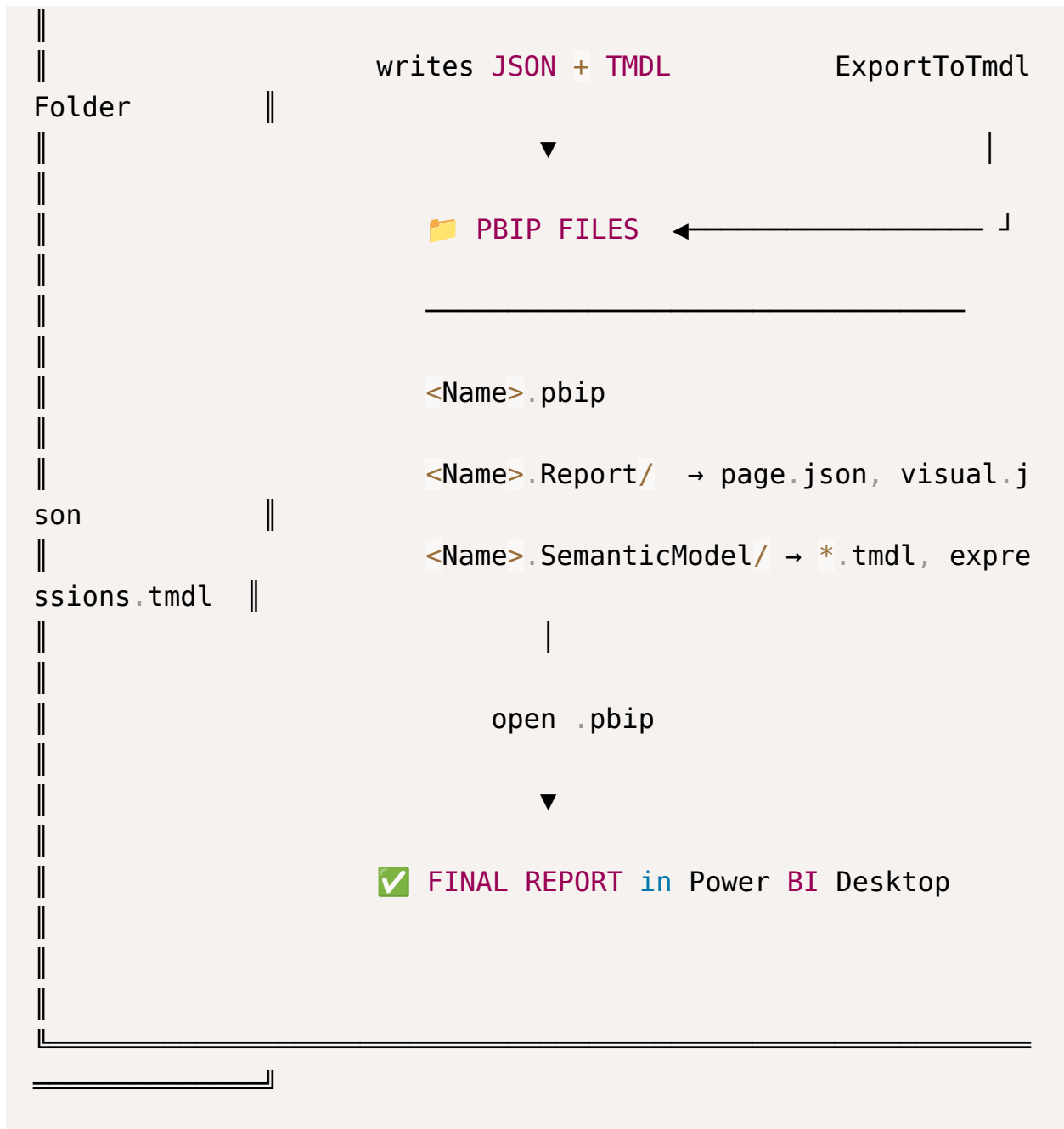




Power BI Report Development using CLAUDE

Object Flow Diagram





📁 **Excalidraw file** (open in VS Code Excalidraw extension or excalidraw.com):

Excalidraw — Collaborative whiteboarding made easy

Excalidraw is a virtual collaborative whiteboard tool that lets you easily sketch diagrams that have a hand-drawn feel to them.

✂ https://excalidraw.com/#json=s5zthvgesNSz_yX04rNpZ,2JaU4YR_0RQozjdecyOWHA



C:\Users\AtikantJain\CLAUDE-COWORK-AG\PowerBIAgent\PowerBI_Pipeline_Flow.excalidraw

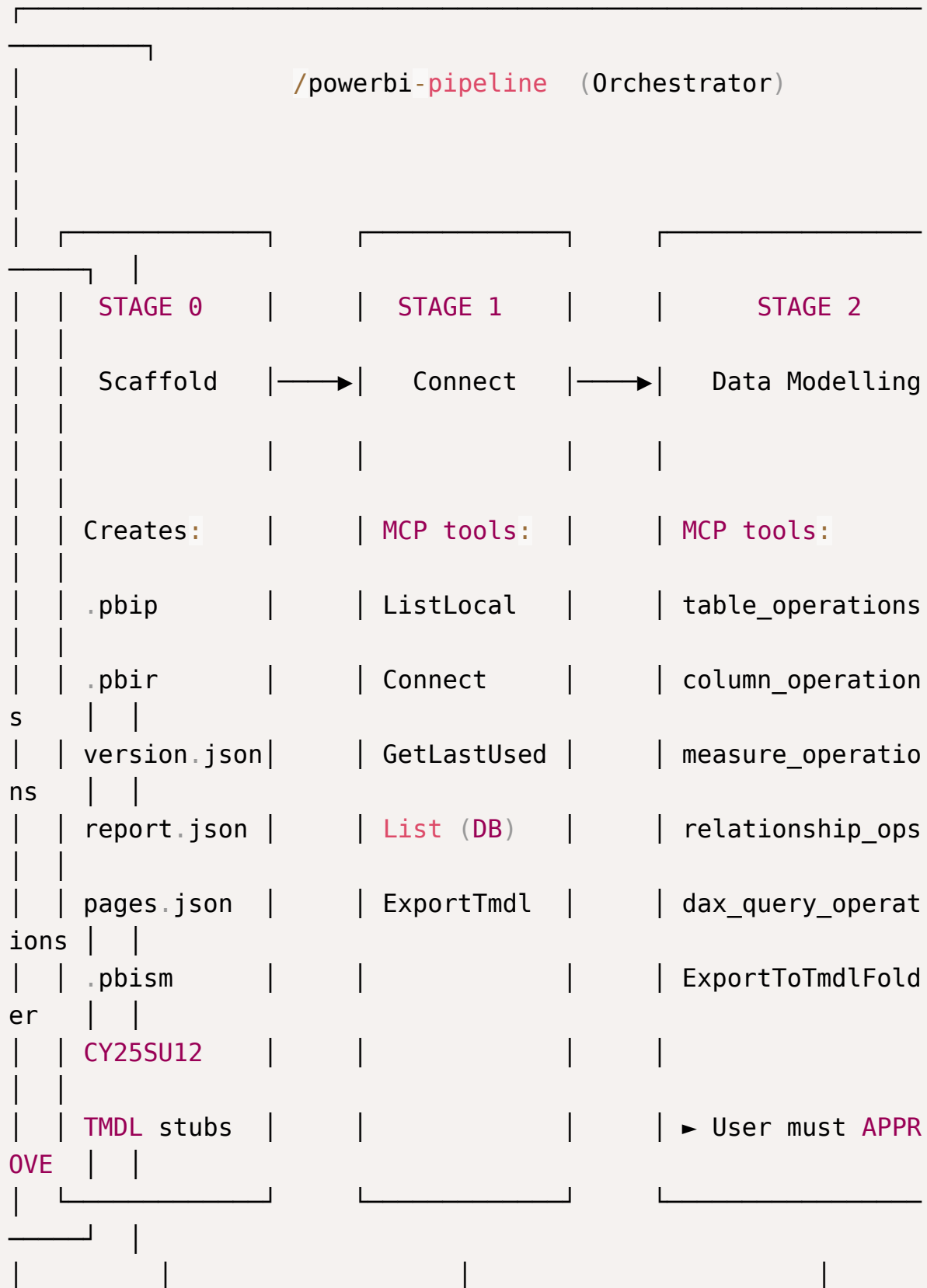
What Is This?

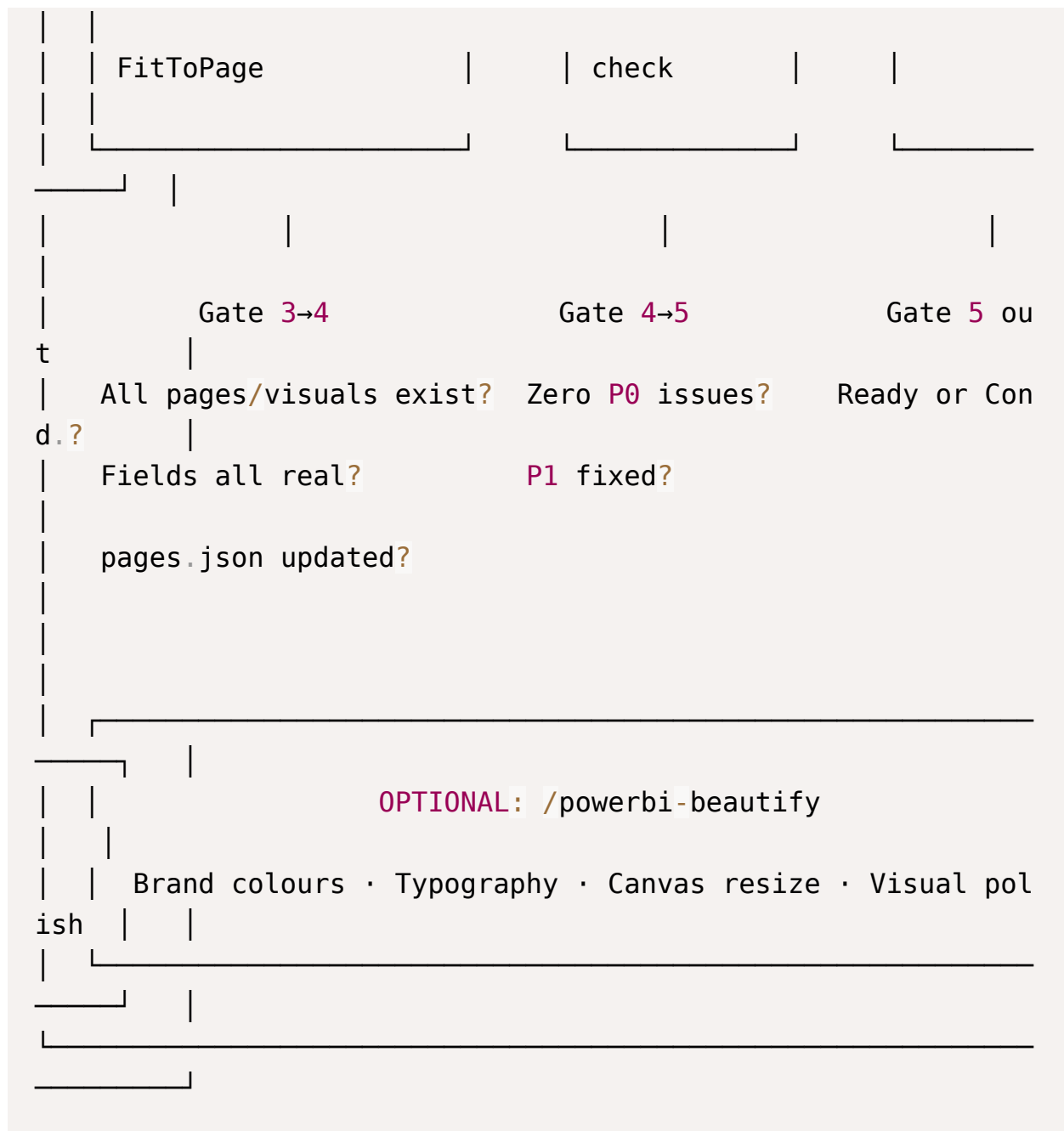
This page documents the **Power BI PBIP End-to-End Development Pipeline** built with Claude Code and the `powerbi-modeling-mcp` MCP server. The pipeline turns a brief and a live Power BI Desktop model into a fully-written `.pbip` project — with semantic measures, report pages, and visual files — that can be opened directly in Power BI Desktop.

The 6 Skills

Skill	Slash Command	Purpose
<code>powerbi-scaffold</code>	<code>/powerbi-scaffold</code>	Creates the blank PBIP folder scaffold
<code>powerbi-connect</code>	<code>/powerbi-connect</code>	Connects to Local Desktop or Fabric workspace
<code>powerbi-datamodelling</code>	<code>/powerbi-datamodelling</code>	Profiles model, creates DAX measures, persists to TMDL
<code>powerbi-reporting</code>	<code>/powerbi-reporting</code>	Writes page.json and visual.json files for all report pages
<code>powerbi-beautify</code>	<code>/powerbi-beautify</code>	Applies brand colours, typography, and visual polish
<code>powerbi-pipeline</code>	<code>/powerbi-pipeline</code>	Orchestrates all above in order, with hard gates between stages

Pipeline Flow Diagram





Stage-by-Stage Breakdown

Stage 0 — Scaffold (**powerbi-scaffold**)

Creates the complete PBIP folder and file scaffold from scratch. Nothing is hardcoded — every path is derived from the project name you provide.

What it creates:

- `<Name>.pbip` — the launcher file Power BI Desktop opens
- `<Name>.Report/definition.pbir` — links the report to its semantic model
- `<Name>.Report/definition/version.json` — schema version metadata
- `<Name>.Report/definition/report.json` — theme, settings, resource packages
- `<Name>.Report/definition/pages/pages.json` — empty page order (filled in Stage 3)
- `<Name>.Report/StaticResources/SharedResources/BaseThemes/CY25SU12.json` — default theme file
- `<Name>.SemanticModel/definition.pbism` — semantic model definition
- TMDL stub files — replaced with real export in Stage 1

Critical rule: `report.json` must NEVER contain `name`, `displayName`, or `description` at the top level — Desktop will refuse to open the file.

Stage 1 — Connect (`powerbi-connect`)

Establishes a live connection to a running Power BI semantic model — either a local Desktop instance or a Fabric workspace model.

Two paths:

- **Local:** Calls `ListLocalInstances` → picks the running Desktop window → `Connect` → `GetLastUsed` to confirm
- **Fabric:** Authenticates → lists workspaces → fuzzy-matches name with user confirmation → `ConnectFabric`

Post-connection export: Runs `ExportToTmdlFolder` to write the real model (including `expressions.tmdl`) into the SemanticModel folder. This is the fix for the "import X matches no exports" error that occurs when M partitions reference named Power Query queries.

Stage 2 — Data Modelling (`powerbi-datamodelling`)

The intelligence layer. Profiles the live model with DAX queries, builds a transformation plan, and only applies it after explicit user approval.

Profile queries run:

- Row counts and date ranges
- Key measure totals and averages
- Top 5 categorical breakdowns

What gets created (example — OutageAnalytics project):

Display Folder	Measures
KPIs	Total Outages, Affected Customers, Avg Duration, Max Duration, Total Customer-Hours Lost
Time Intelligence	MoM Change %, Outages YTD, Cumulative Customers, Weekend Ratio
Outlier Analysis	Duration Q1/Q3/Median/P90, IQR, Upper Fence, Outlier Count, Outlier %
Executive	Impact Score, High Impact Flag, Avg Impact Score, High Impact %

Gate rule: No model changes are made until the user types `approve`. All DAX is validated with `dax_query_operations Validate` before Stage 3.

Stage 3 — Reporting (`powerbi-reporting`)

Writes all report page and visual files. Never uses hardcoded field names — always resolves from the live model first.

Layout grid (1280×720 canvas):

Zone	Position	Use for
Header strip	y=0, h=42	Title / brand bar
Filter strip	y=48, h=38	Slicers
KPI strip	y=93, h=100	Headline number cards
Main chart left	y=200, w=620, h=245	Primary chart
Main chart right	y=200, x=640, h=245	Secondary chart
Bottom left	y=455, w=630, h=255	Trend / detail
Bottom right	y=455, x=640, h=255	Breakdown / table

Valid visual types (selection):

- `clusteredBarChart` — horizontal bar

- `clusteredColumnChart` — vertical column
- `lineChart` — trend line
- `matrix` — pivot/heatmap
- `tableEx` — flat table
- `card` — single KPI number
- `scatterChart` — bubble/scatter
- `slicer` — filter control

`barChart`, `pivotTable`, `columnChart` do NOT exist — they render as broken "install custom visual" prompts.

Strictly forbidden in visual.json: `"filters": [...]` inside the `visual` block. This property is not in the schema and crashes Desktop on file open. TopN filters must be set manually in the Desktop Filters pane.

Stage 4 — Validation (inline in `/powerbi-pipeline`)

Four checks run before evaluation:

1. **DAX re-validation** — every measure used in visuals re-validated
2. **Schema scan** — grep all visual.json files for `filters`, invalid `visualType`, missing `$schema`
3. **Field binding check** — every `Entity` + `Property` pair confirmed to exist in live model
4. **File structure check** — all pages in `pageOrder` have `page.json`, all visual dirs have `visual.json`, `CY25SU12.json` exists

Issue severity model:

Severity	Examples	Action
P0	<code>filters</code> in visual, invalid <code>visualType</code> , missing required file	Block — fix before Stage 5
P1	Missing title/border, wrong format string	Fix in place
P2	Cosmetic, layout spacing	Note for beautify pass

Stage 5 — Evaluation (inline in `/powerbi-pipeline`)

Assesses the completed report against the original brief:

- Does every stated KPI have a visual?
- Does visual density match audience (Executive = 4–8 visuals/page)?
- Do DAX values match profiling queries from Stage 2?
- Are all requested pages built?
- Will the report open cleanly in Desktop?

Output states:  Ready |  Ready with Conditions |  Not Ready

Optional: Beautify (`powerbi-beautify`)

Applies visual polish without touching any data bindings. Can run standalone after reporting or as a post-pipeline step.

What it does:

- Resizes canvas (landscape 1280×720 ↔ portrait 1080×1920)
- Applies brand colour palette to `report.json` data colours
- Updates `visualContainerObjects` on every visual: title font, border colour, background colour
- KPI cards get primary-colour background treatment
- Slicers get subtle border-only style
- Header bar gets full-width brand colour fill

Default palette:

Role	Hex
Primary	<code>#1A73E8</code>
Secondary	<code>#34A853</code>
Accent (warning)	<code>#FDBC04</code>
Accent (negative)	<code>#EA4335</code>

Role	Hex
Canvas (dark)	#202124
Card background	#3C4043
Border	#5F6368

MCP Tools Used

Tool	Used In
connection_operations → ListLocalInstances	Stage 1
connection_operations → Connect	Stage 1
connection_operations → GetLastUsed	Stage 1, Gate checks
database_operations → List	Stage 1
database_operations → ExportToTmdlFolder	Stage 1, Stage 2
table_operations → List	Stage 2, Stage 3
table_operations → GetSchema	Stage 2
measure_operations → List	Stage 2, Stage 3
measure_operations → Create	Stage 2
column_operations → List	Stage 2, Stage 3
column_operations → Update	Stage 2
relationship_operations → List	Stage 2
dax_query_operations → Execute	Stage 2 (profiling)
dax_query_operations → Validate	Stage 2, Stage 4

Two Ways to Invoke

Option A — Full Pipeline (Recommended)

```
/powerbi-pipeline
```

Runs all stages in sequence with hard gates. Cannot skip a stage. User approval is required at Stage 2 before any model changes are applied.

Option B — Individual Stages

```
/powerbi-scaffold      ← run once per new project
/powerbi-connect       ← after Desktop is open with your .pbix/.pbip
/powerbi-datamodelling ← profile + measure creation (requires approval)
/powerbi-reporting     ← write all pages and visuals
/powerbi-beautify      ← apply brand colours and visual polish
```

Useful when resuming a project mid-pipeline or iterating on a single stage.

Real Project: OutageAnalytics

Pipeline was run for the **OutageAnalytics** report connected to **DashboardOff-Claude** (local Power BI Desktop).

Model: Dataset table — 2,356 outage events, Jan–Dec 2020, 8 outage reasons, 46 LGAs

Output: 4-page executive report

Page	Purpose	Visuals
Overview	High-level KPI snapshot + reason/region breakdowns	12
Time Analysis	Day-of-week, time-of-day, heatmap, duration distribution	8
Outlier Analysis	Statistical outliers using IQR method, bubble chart, detail table	9
Impact Intelligence	Composite impact scores, trend by reason, quarterly matrix	8

Measures created: 22 across 4 display folders

Key lessons from this build:

- Always run `ExportToTmdlFolder` after connect — `expressions.tmdl` is critical for M partitions with `Table.Combine`
 - Never use `filters` inside `visual.json` — crashes Desktop
 - Table name `Dataset` requires single-quote wrapping in all DAX:
`'Dataset'[Column]`
 - TopN filters (Top 10 LGAs, Top 20 events, Top 15 by Impact Score) must be set manually in the Desktop Filters pane
-

Skill Files Location

```
C:\Users\AtikantJain\CLAUDE-COWORK-AG\PowerBIAgent\NEW SKILLS
\
├─ powerbi-scaffold\SKILL.md
├─ powerbi-connect\SKILL.md
├─ powerbi-datamodelling\SKILL.md
├─ powerbi-reporting\SKILL.md
├─ powerbi-beautify\SKILL.md
└─ powerbi-pipeline\SKILL.md
```